Integración por Monte Carlo en memoria compartida con OpenMP

Evaluación de programación paralela

Introducción a la computación paralela
Universidad Industrial de Santander · Grupo 28661
Docente: Sergio Augusto Gélvez Cortés

Nancy Liliana Saenz Moreno

Código 2224510 · Semilla usada: 2224510

La idea en una frase. Se lanzan muchos puntos al azar sobre un cuadrado y se cuenta cuántos caen dentro de un cuarto de círculo: esa fracción es $\pi/4$. Como cada punto es independiente, el trabajo se puede repartir entre varios núcleos del procesador con OpenMP para obtener el mismo resultado en menos tiempo.

Todas las ejecuciones se hicieron en mi portátil (AMD Ryzen 5 3450U: 4 núcleos y 8 hilos, Ubuntu en WSL, gcc 15.2.0). Las capturas de pantalla son de esas ejecuciones; los programas, scripts y salidas completas están en el repositorio Git del proyecto.

27 de octubre de 2026

Índice

Parte I. Fundamento del método **2**

Parte II. Implementación en OpenMP **6**

Parte III. Evaluación de desempeño: speedup y ley de Amdahl **11**

Parte IV. Informe sobre su proceso de solución **18**

Anexo. Bitácora de IA **20**

Mis resultados principales (semilla 2224510, $N = 10^8$).

- $\pi \approx 3{,}141668$, con un error de 0,0000756 (el error típico esperado es 0,000164).
- El mismo conteo, **78 541 707 aciertos**, con 1, 2, 4 y 8 hilos: el resultado no depende del número de hilos.
- Speedup: $S(2) = 1{,}84$, $S(4) = 3{,}39$ y $S(8) = 5{,}02$; fracción paralelizable estimada $f = 0{,}940$.

Parte I. Fundamento del método

Se lanzan puntos al azar sobre un cuadrado de lado 1, como si fueran dardos. La fracción que cae dentro del cuarto de círculo es $\pi/4$, así que

$$\pi \approx 4 \times \frac{\text{puntos dentro}}{\text{puntos totales}}.$$

Con los primeros 2000 puntos de mi semilla cayeron 1568 dentro, lo que da $\pi \approx 3{,}136$ (Figura 1). Con 100 millones de puntos da $\pi \approx 3{,}141668$.

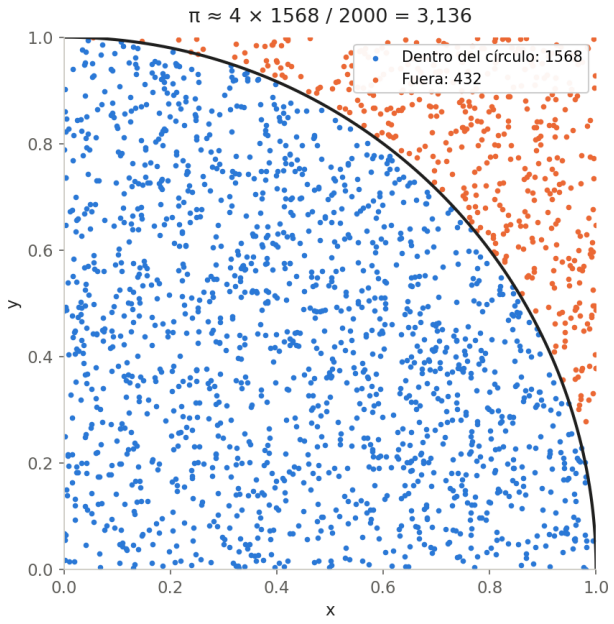


Figura 1. Los primeros 2000 puntos con mi semilla: en azul los que caen dentro del cuarto de círculo y en naranja los que caen fuera.

I.1. Explique con sus propias palabras por qué el error del estimador disminuye en proporción a $1/\sqrt{N}$ y deduzca cuántas muestras adicionales se requieren para reducir el error a la mitad. ¿Qué implicación tiene esto sobre el costo de ganar un dígito de precisión?

¿Por qué $1/\sqrt{N}$? Cada punto es como lanzar una moneda cargada: cae dentro con probabilidad $q = \pi/4 \approx 0{,}785$ o cae fuera. Un solo punto no dice casi nada, pero al promediar muchos, los "errores de suerte" de unos se compensan con los de otros. Esa compensación no es perfecta: cuando se promedian N cosas al azar, la dispersión baja como $\sqrt{N}$, no como N. Por eso el error típico es

$$\text{error} \approx 4\sqrt{\frac{q(1-q)}{N}} \approx \frac{1{,}64}{\sqrt{N}}.$$

Lo comprobé en mi computador (Figura 2). Para cada N repetí el cálculo 30 veces, cada vez con un tramo distinto de números aleatorios, y promedié el error. Mis datos (azul) caen sobre la fórmula

(naranja): con $N = 10^6$ el error medio fue 0,00175 y la fórmula predice 0,00164. Una sola corrida puede tener más o menos suerte (puntos grises); por eso la regla habla del error *típico*.

¿Cuántas muestras más para reducir el error a la mitad? Si el error es $1{,}64/\sqrt{N}$, para que sea la mitad hay que duplicar $\sqrt{N}$, es decir, **multiplicar N por 4**: hay que agregar $3N$ **muestras más** de las que ya se tenían.

¿Qué implica para ganar un dígito? Un dígito más es un error 10 veces menor, y eso exige **100 veces más muestras** y 100 veces más tiempo. Si 10^8 puntos tardan cerca de 1,3 s en mi portátil, ganar un dígito costaría unos 2 minutos y otro dígito más, unas 3,6 horas. Monte Carlo es fácil de programar, pero **cada dígito de precisión sale cada vez más caro**.

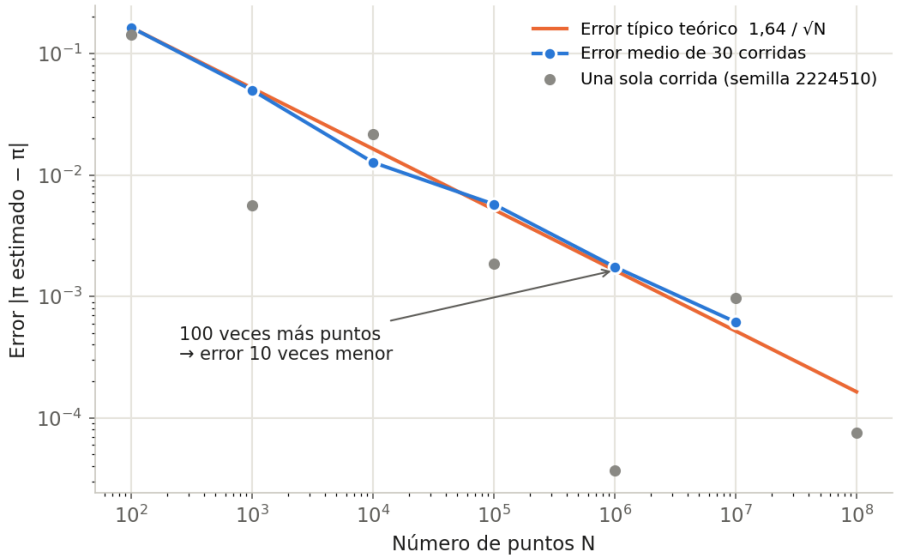


Figura 2. Error frente al número de puntos (escala logarítmica): la fórmula $1{,}64/\sqrt{N}$, el promedio de 30 corridas y una sola corrida con mi semilla.

I.2. Justifique por qué la integración por Monte Carlo se clasifica como un problema embarazosamente paralelo: analice las dependencias entre las muestras (o su ausencia) y determine qué datos deben ser compartidos y cuáles deben ser privados cuando el muestreo se reparte entre varios hilos de ejecución.

¿Por qué "embarazosamente paralelo"? Porque se reparte casi sin esfuerzo: **ningún punto depende de otro**. Para saber si el punto 500 cae dentro no hace falta conocer el 499 ni ningún otro. No hay que esperar resultados ajenos ni intercambiar datos mientras se calcula. Lo único que se junta es el **número de aciertos**, una sola vez y al final.

¿Qué se comparte y qué es privado? Según lo visto en clase, lo declarado fuera de la región paralela es compartido y lo declarado dentro es privado (Figura 3):

Dato	Tipo	Por qué
N y la semilla base	Compartidos	Todos los leen y nadie los cambia
Estado del generador	**Privado**	Si fuera compartido, los hilos se pisarían al pedir números
x, y y la variable del ciclo	Privados	Cada hilo calcula sus propios puntos
Contador de aciertos	Privado y **sumado al final**	Con `reduction(+:dentro)` cada hilo cuenta en su copia y OpenMP las suma

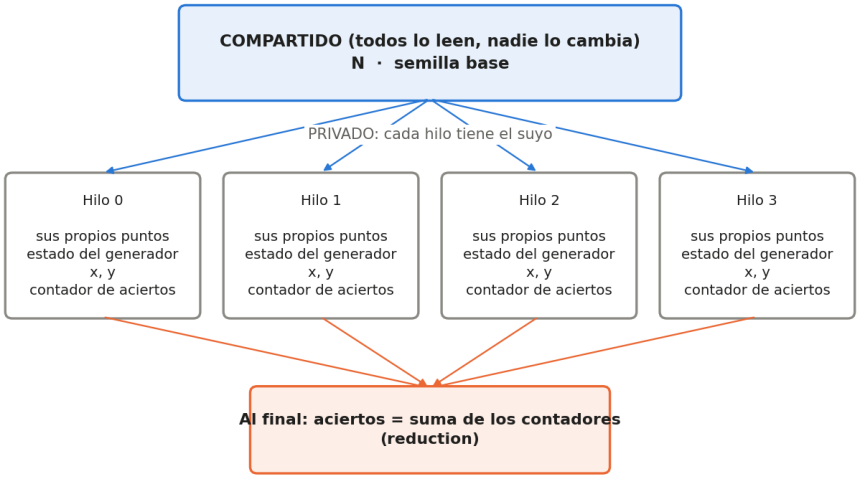


Figura 3. Lo que ven todos los hilos (compartido), lo que tiene cada uno (privado) y la suma final.

I.3. Demuestre que repartir las N muestras entre p hilos y sumar los aciertos parciales de cada uno produce el mismo estimador que la versión secuencial. Con el mismo N total, ¿qué precisión estadística cabe esperar respecto de la ejecución con un solo hilo? Justifique.

Demostración. La versión secuencial cuenta los aciertos uno por uno: $\text{dentro} = a_1 + a_2 + \cdots + a_N$, donde a_i vale 1 si el punto i cayó dentro y 0 si no. Si los N puntos se reparten en p grupos sin repetir ni dejar ninguno por fuera, cada hilo k obtiene un parcial d_k, y

$$d_1 + d_2 + \cdots + d_p = a_1 + a_2 + \cdots + a_N = \text{dentro},$$

porque es **la misma suma agrupada de otra forma**: el orden no cambia una suma de enteros. Entonces $4 \cdot \text{dentro}/N$ es **exactamente el mismo**. Como el contador es entero, ni siquiera aparecen las diferencias de redondeo que vimos en clase al reducir números decimales.

Lo comprobé con $N = 10^8$ partido en 4 tramos (Figura 4): $19\,636\,256 + 19\,634\,338 + 19\,636\,316 + 19\,634\,797 = \mathbf{78\,541\,707}$, el mismo conteo del programa secuencial.

Precisión con p hilos: la misma que con uno. La precisión depende de **cuántos puntos se usan** (N), no de cuántos hilos los calculan: con $N = 10^8$ el error típico es 0,000164 con 1 hilo o con 8. Lo único

que cambia es el **tiempo**. Esto vale si cada hilo usa puntos distintos: si todos usaran la misma semilla, solo habría N/p puntos diferentes y el error sería $\sqrt{p}$ veces mayor (lo muestro en II.3).

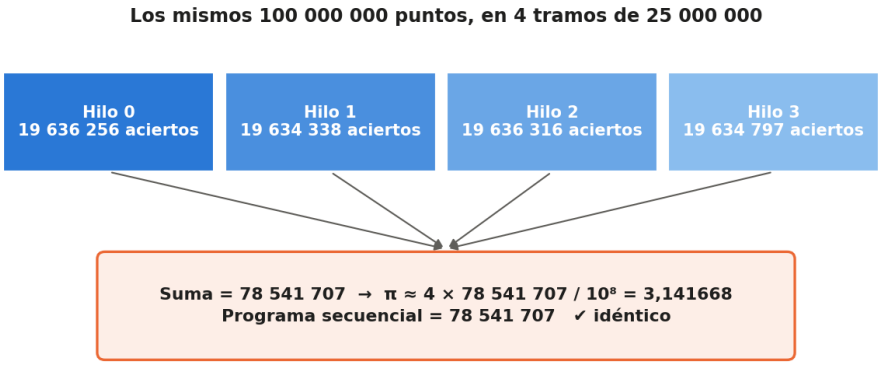


Figura 4. Los 10^8 puntos en 4 tramos: la suma de los parciales es igual al conteo secuencial.

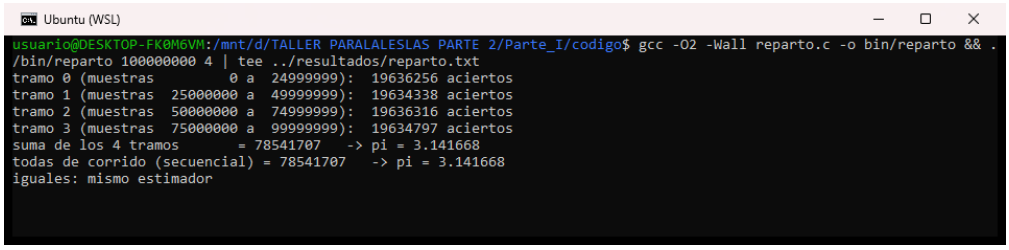


Figura 5. Ejecución real en mi computador del reparto en 4 tramos.

Parte II. Implementación en OpenMP

El enunciado trae la semilla 20261018, pero el profesor indicó que la semilla de cada estudiante es su código, así que en todos los programas usé **2224510**.

II.1. Compile y ejecute la versión secuencial con $N = 10^8$. Verifique que la estimación es consistente con el valor de π (un error del orden de 10^{-4} es normal; explique de dónde sale ese orden de magnitud). Registre el tiempo de ejecución $T(1)$ con `omp_get_wtime()` y consérvelo para la parte III.

Compilé el programa del enunciado con `gcc -O2 montecarlo.c -o montecarlo -lm`, cambiando solo la semilla. Imprimió $\pi \approx 3{,}141668$: 78 541 707 de los 10^8 puntos cayeron dentro. El valor real es 3,141593, así que el error es de **0,0000756**.

¿De dónde sale el orden de 10^{-4}? El error típico es $1{,}64/\sqrt{N}$ (Parte I). Con $N = 10^8$, $\sqrt{N} = 10\,000$ y el error esperado es $\approx 0{,}000164$, del orden de 10^{-4}. Mi error es menor, así que la estimación es consistente.

Tiempo: medí el ciclo con `omp_get_wtime()` en 7 corridas (de 1,1999 s a 1,2743 s). La mediana es $T(1) = 1{,}2288$ **s**. En la Parte III repito esta medida junto con los tiempos de varios hilos, para que la comparación sea justa.

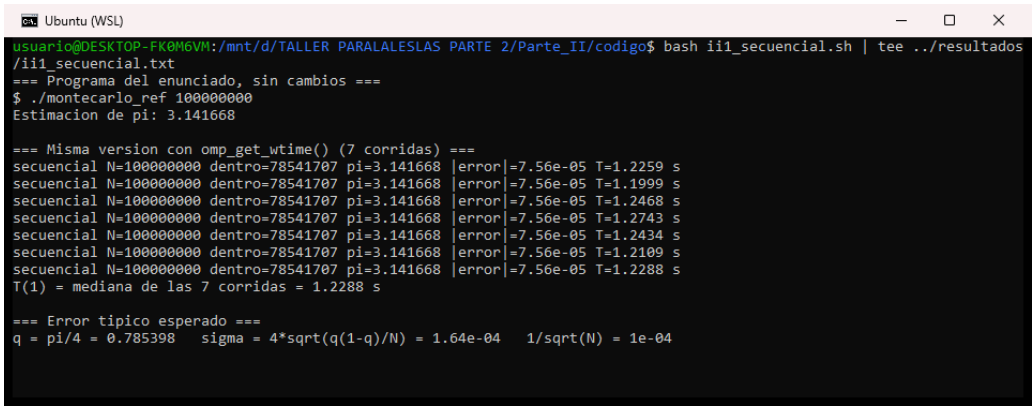


Figura 6. II.1: el programa del enunciado da $\pi = 3{,}141668$; con `omp_get_wtime()`, mediana $T(1) = 1{,}2288$ s.

II.2. Implemente la versión paralela con OpenMP. Su programa debe:

- **usar una directiva** `omp parallel for` **con la cláusula de reducción adecuada sobre el acumulador de aciertos;**
- **permitir elegir el número de hilos** p **(con** `omp_set_num_threads` **o** `OMP_NUM_THREADS`**);**
- **garantizar que cada hilo disponga de un flujo de números aleatorios independiente y sin solapamiento con el de los demás. Como semilla base use la que su docente le asignó (las semillas se publican por estudiante) y espacie las de cada hilo al menos** N/p **valores. El resultado debe ser reproducible: con las mismas semillas, el programa debe producir la misma estimación cualquiera que sea el valor de** p**.**

Mantuve el programa original casi igual (el mismo `rand_r`, la misma prueba $x^2 + y^2 \leq 1$ y el mismo ciclo). Lo nuevo es que los 10^8 puntos se dividen en **1000 bloques de 100 000** y OpenMP reparte los bloques entre los hilos:

```
#pragma omp parallel for reduction(+:dentro) schedule(static)
for (long b = 0; b < nbloques; b++) {
    unsigned int estado = saltar(semilla, b * BLOQUE * 6);
    /* aqui va el mismo ciclo del programa original */
}
```

- **Reducción:** `reduction(+:dentro)`; cada hilo cuenta en su copia y al final se suman.
- **Elegir los hilos:** como segundo argumento, por ejemplo `./montecarlo 100000000 4` (el programa llama a `omp_set_num_threads`), o con la variable `OMP_NUM_THREADS=4`.
- **Flujos sin solapamiento:** los números salen de **una sola lista fija** que empieza en 2224510. La función `saltar` calcula en pocos pasos el estado del generador en cualquier punto de la lista, y así cada bloque arranca en "su" parte (Figura 7). Es como repartir un libro por capítulos. Con 4 hilos, cada uno empieza 25 000 000 muestras (N/p) después del anterior; como cada muestra usa 6 valores del generador, la separación es de 150 millones de valores, muy por encima del mínimo de N/p.
- **Mismo resultado con cualquier** p**:** el conteo fue **78 541 707 con 1, 2, 4 y 8 hilos**, en todas las repeticiones, igual al secuencial. Con 8 hilos el tiempo bajó de 1,33 s a 0,22 s.



Figura 7. Cómo se reparte el trabajo con 4 hilos: cada uno toma un tramo distinto de la misma lista.

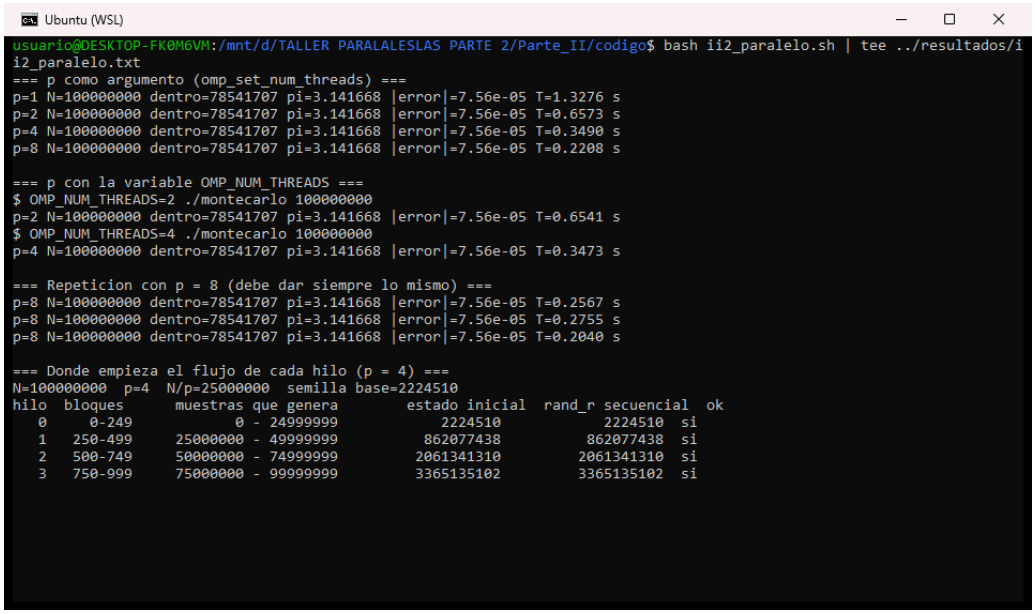


Figura 8. II.2: el mismo conteo con 1, 2, 4 y 8 hilos (argumento y `OMP_NUM_THREADS`) y la verificación del estado inicial de cada hilo.

II.3. Explique por qué no es correcto que todos los hilos llamen a la función `rand()` compartiendo el estado global del generador. Su respuesta debe cubrir al menos dos problemas distintos: uno de condición de carrera sobre el estado interno, y otro de correlación entre secuencias si cada hilo inicializara el generador con la misma semilla. Indique qué consecuencias tendría cada uno sobre la estimación de π y sobre el error reportado.

`rand()` guarda su posición en la lista en **una sola variable escondida** que comparten todos los hilos. Escribí programas que cometen cada problema a propósito.

Problema 1: condición de carrera sobre el estado. Es como si 8 personas escribieran a la vez en el mismo cuaderno: se pisan. Dos hilos pueden sacar el mismo número o borrar el avance del otro.

- Con `rand()`, la biblioteca le pone un candado y los hilos se turnan: con 8 hilos tardó **entre 11,2 y 12,1 s, contra 1,56 s con uno**, y el conteo cambió en cada corrida (78 540 004, 78 539 170 y 78 538 732).
- **Sobre π:** deja de ser reproducible y puede quedar sesgado (números repetidos o saltados).
- **Sobre el error reportado:** la fórmula supone 10^8 puntos independientes; si hay repetidos, hay menos puntos reales y el error verdadero es mayor que el reportado. Con un estado compartido sin candado, una corrida tuvo error 0,000415, más del doble de lo esperado.

Problema 2: correlación por la misma semilla. Si cada hilo tiene su generador pero todos empiezan igual, **todos generan los mismos puntos**.

- Con 2, 4 y 8 hilos el resultado fue **idéntico al del secuencial con solo $N/2$, $N/4$ y $N/8$ puntos**. Con 8 hilos, $\pi = 3{,}142351$, lo mismo que 12,5 millones de puntos.
- **Sobre π:** depende de p y en realidad solo hay N/p puntos distintos.
- **Sobre el error reportado:** el real crece como $\sqrt{p}$. Con 8 hilos fue 0,000759, pero con la fórmula para 10^8 puntos se reportaría $\pm 0{,}000164$: casi 3 veces más precisión de la real.

Mi versión evita ambos: cada bloque tiene su propio estado y empieza en un lugar distinto de la lista.

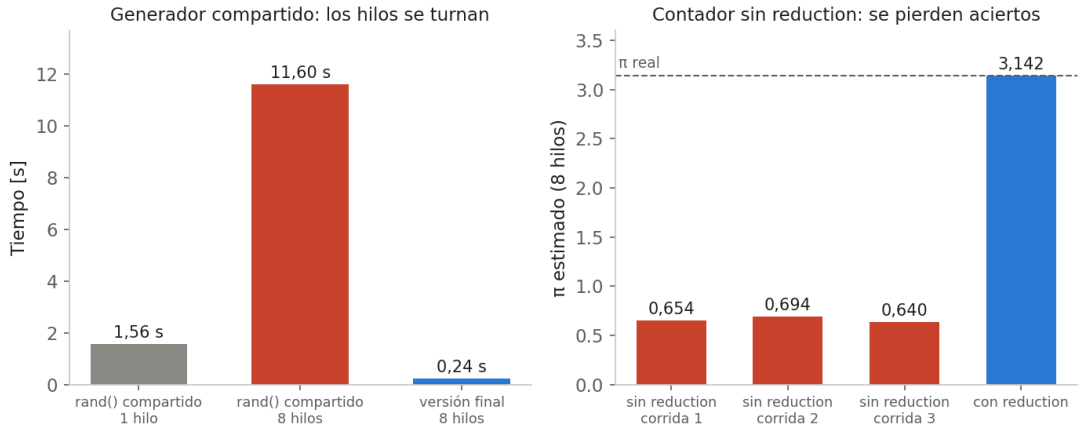


Figura 9. Izquierda: con `rand()` compartido, 8 hilos tardan mucho más que uno. Derecha: sin `reduction`, π sale muy por debajo del real (II.4).

II.4. Analice los atributos de compartición de datos de su región paralela: enumere las variables que OpenMP trata como compartidas y como privadas por defecto en su código, explique la cláusula que usó para el acumulador y describa qué ocurriría si el acumulador se actualizara dentro del ciclo sin esa cláusula (diagnostique el tipo de defecto y su efecto sobre el resultado).

Para comprobarlo con mi código compilé la región con `default(none)`, que obliga a declarar todo: el compilador dio un error por cada variable compartida, `nbloques`, `semilla` y `N` (Figura 11).

Variable	Tipo por defecto	Por qué
`N`, `nbloques`, `semilla`	Compartidas	Declaradas antes del ciclo; solo se leen
`b`	Privada	Es la variable del `omp parallel for`
`estado`, `fin`, `i`, `x`, y `dentro`	Privadas	Declaradas dentro del ciclo
	Privada gracias a `reduction`	Es el contador de aciertos

Cláusula usada: `reduction(+:dentro)`**.** Cada hilo recibe una copia que empieza en 0, cuenta ahí sus aciertos y al final OpenMP suma las copias. Es más rápida que `critical` o `atomic` en cada acierto, porque los hilos se coordinan una sola vez.

Sin la cláusula: `dentro++` son tres pasos (leer, sumar, guardar). Si dos hilos leen el mismo valor a la vez, ambos guardan lo mismo y se pierde un acierto (Figura 10). Es una **condición de carrera**. Lo probé: con 1 hilo da bien (por eso no se nota sin paralelismo), pero con 8 hilos se perdió casi el 80 % de los aciertos y π salió **0,654; 0,694 y 0,640**, siempre **menor** que el real y distinto en cada ejecución.

Tiempo	Hilo 0	Hilo 1	dentro en memoria
1	lee dentro (100)		100
2		lee dentro (100)	100
3	suma 1 → 101	suma 1 → 101	100
4	escribe 101		101
5		escribe 101	**101 ← se perdió un acierto**

Figura 10. Cómo se pierde un acierto cuando dos hilos actualizan el mismo contador a la vez.

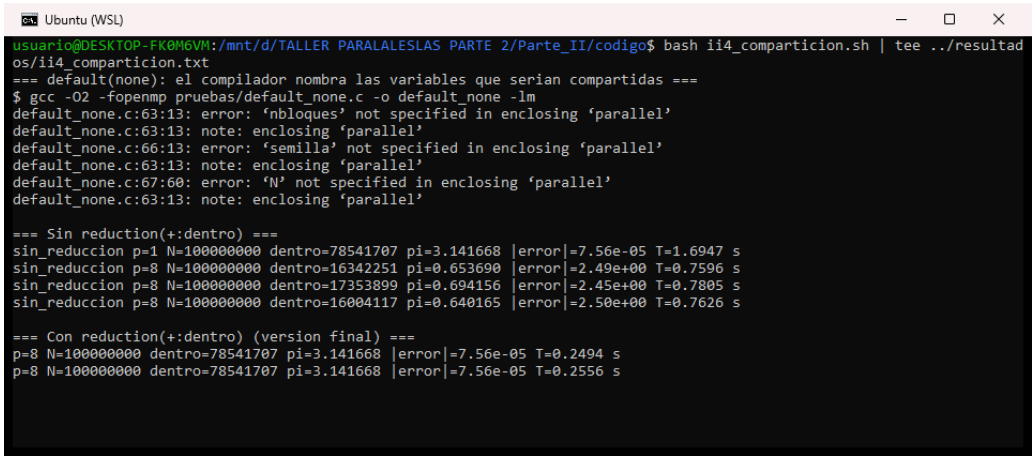


Figura 11. II.4: `default(none)` nombra las variables compartidas; sin `reduction` π sale entre 0,64 y 0,69.

Parte III. Evaluación de desempeño: speedup y ley de Amdahl

El *speedup* con p hilos es $S(p) = T(1)/T(p)$ y la *eficiencia* es $E(p) = S(p)/p$. Si una fracción f del trabajo es paralelizable, la *ley de Amdahl* da el speedup máximo:

$$S(p) = \frac{1}{(1-f) + f/p}.$$

Protocolo experimental. Con $N = 10^8$ fijo, mida $T(p)$ para $p \in \{1,2,4,8\}$. Para cada valor de p realice al menos cinco ejecuciones y registre la mediana de los tiempos; justifique por qué la mediana es más apropiada que el promedio en este experimento.

Equipo: AMD Ryzen 5 3450U con **4 núcleos físicos y 2 hilos por núcleo** según `lscpu` (Figura 13), conectado al cargador. **Cómo medí:** 7 rondas; en cada una corrí el programa secuencial y luego el paralelo con 1, 2, 4 y 8 hilos, uno tras otro, para que todos sufrieran las mismas condiciones. $T(1)$ es el programa secuencial.

p (hilos)	Mediana de $T(p)$ [s]	$S(p) = T(1)/T(p)$	$E(p) = S(p)/p$
1	1,2549	1,000	1,000
2	0,6826	1,838	0,919
4	0,3700	3,392	0,848
8	0,2499	5,022	0,628

Tabla 1. Mediana de 7 ejecuciones con $N = 10^8$. Las 35 ejecuciones dieron 78 541 707 aciertos.

En II.1 medí $T(1) = 1{,}2288$ s; aquí uso el valor medido junto con los demás tiempos (1,2549 s), para comparar en las mismas condiciones. La diferencia es solo del 2 %.

¿Por qué la mediana? A veces el sistema interrumpe una ejecución y la vuelve más lenta (nunca más rápida). En mis datos pasó: en la ronda 4 una corrida secuencial tardó **12,41 s** cuando lo normal era 1,25 s (Figura 12). Con el promedio, $T(1)$ sería 2,84 s y $S(4)$ saldría 7,7: más rápido que 4 núcleos ideales, algo imposible. La mediana toma el valor del medio y esos casos raros no la mueven.

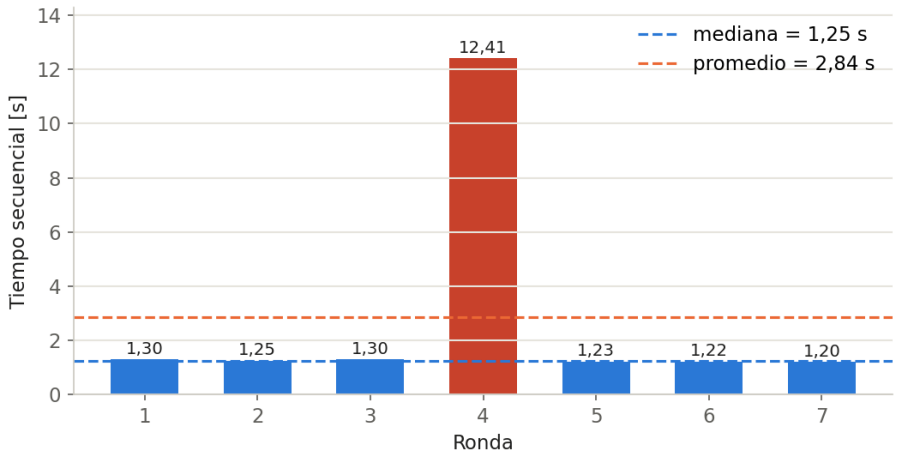


Figura 12. Tiempo secuencial en cada ronda: una sola corrida lenta dispara el promedio, pero no la mediana.

```
Ubuntu (WSL)
usuario@DESKTOP-FK0M6VM:/mnt/d/TALLER PARALALESLAS PARTE 2/Parte_III/codigo$ bash compilar.sh | tee ../resultados/comp
ilacion.txt
gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0
$ gcc -O2 -fopenmp montecarlo_secuencial.c -o montecarlo_secuencial -lm
$ gcc -O2 -fopenmp -Wall -Wextra montecarlo.c -o montecarlo -lm
$ gcc -O2 -fopenmp -Wall sobrecosto.c -o sobrecosto
(todo compilo sin advertencias)

$ lscpu | grep -E 'Model name|^CPU\(s\)|Thread|Core|Socket|MHz|L1d|L2|L3'
CPU(s):                               8
Model name:                           AMD Ryzen 5 3450U with Radeon Vega Mobile Gfx
Thread(s) per core:                   2
Core(s) per socket:                   4
Socket(s):                            1
L1d cache:                            128 KiB (4 instances)
L2 cache:                             2 MiB (4 instances)
L3 cache:                             4 MiB (1 instance)
```

Figura 13. Datos del procesador con `lscpu`: 4 núcleos y 2 hilos por núcleo.

```
Ubuntu (WSL)
usuario@DESKTOP-FK0M6VM:/mnt/d/TALLER PARALALESLAS PARTE 2/Parte_III/codigo$ bash tiempos.sh | tee ../resultados/tiemp
os_pantalla.txt
ronda     sec     p=1     p=2     p=4     p=8
  1    1.2964  1.2099  0.6810  0.3566  0.2515
  2    1.2549  1.2349  0.6585  0.3879  0.2312
  3    1.2969  1.2081  0.7041  0.3700  0.2651
  4   12.4096  1.2726  0.6826  0.3864  0.2896
  5    1.2264  1.2493  0.7419  0.3449  0.2499
  6    1.2226  1.2428  0.6903  0.3747  0.2455
  7    1.2041  1.2475  0.6470  0.3597  0.2377

1 (secuencial)   mediana 1.2549  prom 2.8444  min 1.2041  max 12.4096   S=1.000  E=1.000
p=1 (paralelo)   mediana 1.2428  prom 1.2379  min 1.2081  max 1.2726   S=1.010  E=1.010
p=2 (paralelo)   mediana 0.6826  prom 0.6865  min 0.6470  max 0.7419   S=1.838  E=0.919
p=4 (paralelo)   mediana 0.3700  prom 0.3686  min 0.3449  max 0.3879   S=3.392  E=0.848
p=8 (paralelo)   mediana 0.2499  prom 0.2529  min 0.2312  max 0.2896   S=5.022  E=0.628

f (de S(4) = 3.392) = (1 - 1/S)/(1 - 1/4) = 0.940   ->  1 - f = 0.060
Karp-Flatt: e(2) = 0.088   e(4) = 0.060   e(8) = 0.085
Conteos distintos en las 35 corridas: 78541707
```

Figura 14. Las 7 rondas de medición, las medianas, $S(p)$, $E(p)$, f y Karp–Flatt.

III.1. Aplique la ley de Amdahl a los siguientes casos y interprete cada resultado: (a) $f = 0{,}98$, $p = 8$; (b) $f = 0{,}90$, $p = 8$; (c) $f = 0{,}98$ y p creciendo sin límite. ¿Qué le enseña el caso (c) sobre la fracción secuencial de un programa?

Caso	Cálculo	Speedup	Qué significa
(a) $f = 0{,}98$, $p = 8$	$1/(0{,}02 + 0{,}98/8) = 1/0{,}1425$	**7,02**	Con solo 2 % secuencial, 8 núcleos rinden como 7 ($E = 0{,}88$)
(b) $f = 0{,}90$, $p = 8$	$1/(0{,}10 + 0{,}90/8) = 1/0{,}2125$	**4,71**	Con 10 % secuencial se desperdicia casi la mitad ($E = 0{,}59$)
(c) $f = 0{,}98$, $p \to \infty$	$1/(0{,}02 + 0) = 1/0{,}02$	**50**	Ni con infinitos núcleos pasa de 50 veces

Lo que enseña (c): la parte secuencial pone un **techo** de $1/(1-f)$, por más núcleos que haya (Figura 15). Con 2 % secuencial el techo es 50; con 10 %, solo 10. Para acelerar un programa, **reducir la parte secuencial vale más que comprar núcleos**.

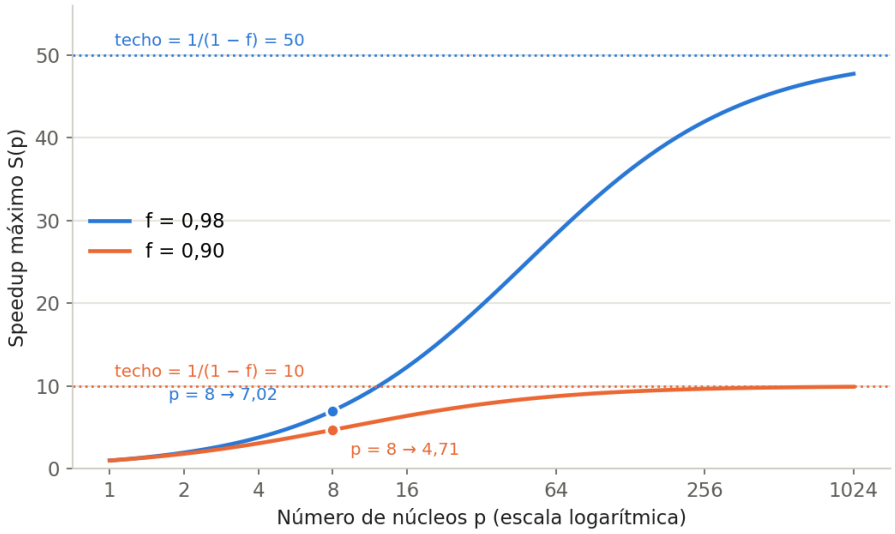


Figura 15. Ley de Amdahl para $f = 0{,}98$ y $f = 0{,}90$: las curvas se aplanan contra su techo.

III.2. Con sus mediciones, estime la fracción secuencial f de su programa: despeje f de la ley de Amdahl a partir del speedup medido con $p = 4$ y muestre el procedimiento completo. Si lo desea, contraste su resultado con la métrica de Karp–Flatt, $e = \dfrac{1/S(p) - 1/p}{1 - 1/p}$, y explique qué mide.

Con $S(4) = 3{,}392$:

$$\frac{1}{S} = (1-f) + \frac{f}{4} = 1 - 0{,}75f \quad \Rightarrow \quad f = \frac{1 - 1/S}{0{,}75} = \frac{1 - 0{,}2948}{0{,}75} = \frac{0{,}7052}{0{,}75} = \mathbf{0{,}940}.$$

El **94 %** del programa se comporta como paralelizable y la **fracción secuencial es** $1 - f = 0{,}060$ (6 %).

Karp–Flatt mide la fracción secuencial "aparente", calculada con lo que realmente se midió. Me dio $e(2) = 0{,}088$, $e(4) = 0{,}060$ (igual a $1 - f$, porque es la misma cuenta) y $e(8) = 0{,}085$. Si fuera constante habría de verdad un 6 % de código imposible de repartir; como **cambia con** p, el freno no es un pedazo fijo de código, sino costos y ruido que dependen del número de hilos. De hecho, lo realmente secuencial de mi programa es mínimo: crear los 4 hilos cuesta 0,57 ms de 370 ms (0,15 %).

III.3. Compare su speedup medido con el ideal lineal $S(p) = p$ y con el predicho por la ley de Amdahl usando su f estimado. Explique las discrepancias que observe. Considere en su análisis, entre otros factores, la creación y destrucción de hilos, el costo de la reducción, los límites de ancho de banda de memoria y la jerarquía de caché, y el efecto de la frecuencia variable del procesador y del hiperprocesamiento.

p	Ideal	Amdahl ($f = 0{,}940$)	Medido
2	2	1,89	1,84
4	4	3,39	3,39
8	8	5,64	5,02

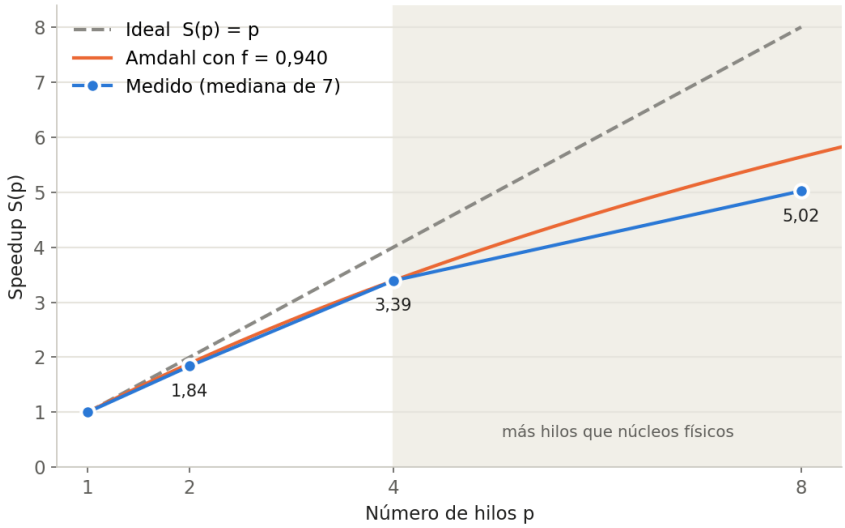


Figura 16. Speedup medido frente al ideal y a Amdahl. La zona sombreada tiene más hilos que núcleos físicos.

Con 8 hilos el programa va 5,02 veces más rápido, lejos del ideal (8) y por debajo de Amdahl (5,64): Amdahl supone un freno fijo, y en mi equipo el freno crece cuando hay más hilos que núcleos. Revisé cada factor con mediciones:

- **Crear y destruir hilos** (Figura 17): 0,36 ms con 2, 0,57 ms con 4 y **13,6 ms con 8**. Con 4 hilos es el 0,15 % del cálculo; con 8 ya es el **5,4 %**.
- **Reducción:** cada hilo suma una sola vez al final; medí entre 1 y 2,3 microsegundos. Despreciable.
- **Memoria y caché:** no influyen. No hay arreglos: cada hilo usa unas pocas variables (estado, x, y, contador) que caben en su caché L1 de 32 KiB.
- **Frecuencia variable** (Figura 18): con 1 hilo el procesador iba a unos **3010 MHz**, con 4 a **2912 MHz** y con 8 a **2572 MHz**. Con un núcleo se acelera (turbo); con todos ocupados, baja: un 3 % con 4 hilos y un **15 %** con 8. La medición tiene ruido: la lectura con 2 hilos salió anómala y no la uso.
- **Hiperprocesamiento:** con 8 hilos hay 2 por núcleo físico y comparten sus recursos. Por eso pasar de 4 a 8 hilos subió la velocidad solo un 48 % (de 3,39 a 5,02).
- **¿Y con 4 hilos?** La frecuencia (3 %) y la creación de hilos (0,15 %) no explican toda la distancia entre 3,39 y 4. Probé `schedule(dynamic)` por si unos hilos esperaban a otros, y **no mejoró** (0,456 s contra 0,412 s; Figura 19), así que descarté esa causa. El resto lo atribuyo a que Windows y Ubuntu (WSL) comparten los mismos 4 núcleos con mi programa.

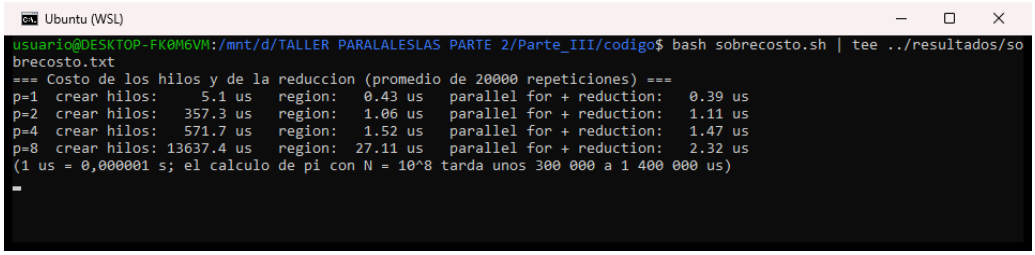


Figura 17. Costo de crear los hilos, de una región paralela y de la reducción (1 us = una millonésima de segundo).

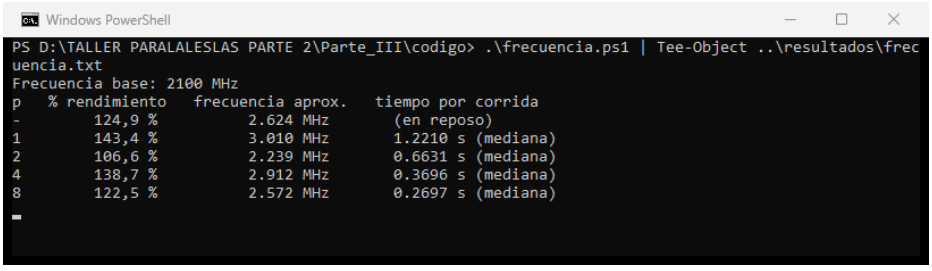


Figura 18. Frecuencia del procesador medida con Windows mientras corría el programa.

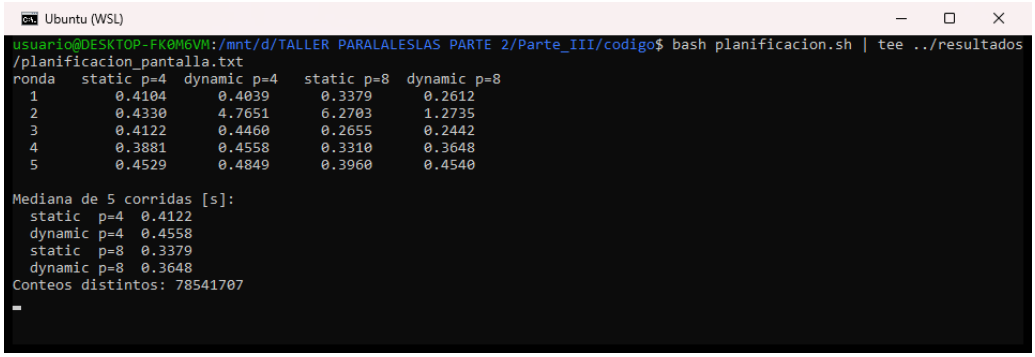


Figura 19. `schedule(static)` frente a `schedule(dynamic)`: dynamic no fue más rápido.

III.4. Suponga que le ofrecen tiempo de cómputo en una máquina con 64 núcleos. Con su fracción f estimada, ¿qué speedup máximo podría obtener y en qué se diferenciaría de crecer el número de muestras N? Reflexione: aumentar p con N fijo, ¿mejora la precisión del resultado o solo el tiempo de cómputo? ¿Qué mejora cada una de las dos variables?

$$S(64) = \frac{1}{0{,}060 + 0{,}940/64} = \frac{1}{0{,}060 + 0{,}0147} = \mathbf{13{,}4}, \qquad \text{techo} = \frac{1}{0{,}060} \approx 16{,}7.$$

Con 64 núcleos el programa iría solo unas 13 veces más rápido (eficiencia del 21 %). Mi f sale de un portátil con turbo, hiperprocesamiento y Windows de fondo; en un servidor el resultado podría ser mejor.

p y N mejoran cosas distintas (Figura 20):

- **Más hilos con el mismo N solo mejora el tiempo:** con 1, 2, 4 y 8 hilos el resultado fue idéntico (78 541 707 aciertos) y el tiempo bajó de 1,30 s a 0,25 s.
- **Más muestras mejora la precisión, pero cuesta tiempo:** el error esperado pasa de 0,0016 (10^6 puntos) a 0,00008 (4×10^8); con 4×10^8 mi error fue 0,0000091, pero el tiempo subió de 0,27 s a 1,10 s.

En resumen, p decide **qué tan rápido** y N decide **qué tan bueno**. Lo ideal es combinarlos: con 64 núcleos conviene usar muchas más muestras en el mismo tiempo. Mi programa tiene un límite para eso: `rand_r` repite su lista después de unos 7×10^8 puntos.

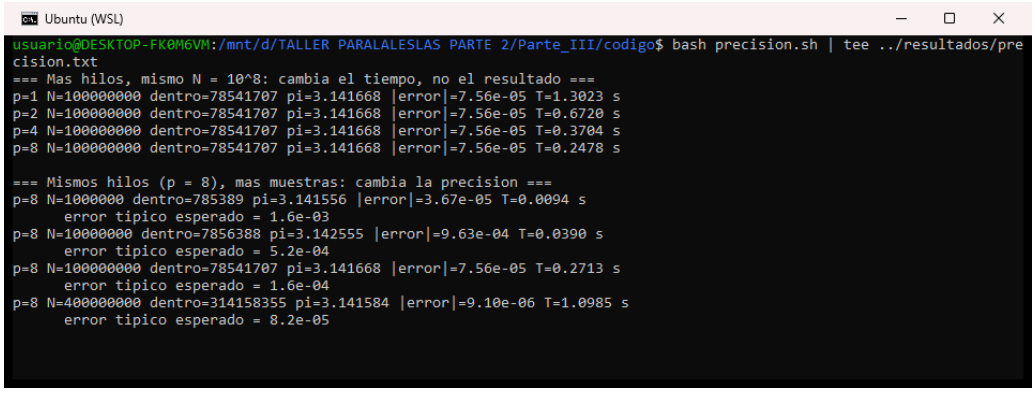


Figura 20. Arriba: más hilos, mismo resultado y menos tiempo. Abajo: más muestras, menos error y más tiempo.

Parte IV. Informe sobre su proceso de solución

Redacte un informe breve, de una a dos páginas, en el que usted explique el trabajo que realizó para llegar a su solución. No se trata de repetir la teoría: se califica la claridad con la que usted da cuenta de su propio proceso. Su informe debe describir, como mínimo:

1. **las decisiones de diseño que tomó y por qué las tomó: estructura del programa, cláusulas de OpenMP empleadas y manejo de las semillas de los generadores de cada hilo;**
2. **los problemas que aparecieron durante el desarrollo y cómo los diagnosticó y corrigió (por ejemplo: resultados distintos en cada ejecución, estimaciones sesgadas, condiciones de carrera) y qué evidencia le permitió identificar cada uno;**
3. **el método que usó para validar la correctitud de sus resultados, tanto la estimación de π como la reproducibilidad con distinto número de hilos;**
4. **la conexión explícita entre su informe y sus datos: cite valores concretos de su tabla (por ejemplo, por qué su mediana con $p = 4$ resultó como resultó) en lugar de descripciones genéricas que podrían aplicarse a cualquier entrega;**
5. **una reflexión final sobre lo que aprendió y sobre lo que haría de manera diferente en un próximo trabajo paralelo.**

1. Decisiones de diseño. Quise cambiar lo menos posible el programa del enunciado: el mismo ciclo, el mismo `rand_r` y la misma prueba $x^2 + y^2 \leq 1$. Lo nuevo es que los 10^8 puntos se dividen en 1000 bloques de 100 000 y cada vuelta del `omp parallel for` hace un bloque. Usé `reduction(+:dentro)` para que cada hilo cuente en su copia y se sume una vez al final (más barato que `critical` o `atomic`), `schedule(static)` porque todos los bloques cuestan lo mismo, y el número de hilos como argumento o con `OMP_NUM_THREADS`. Para las semillas, como el resultado debe ser igual con cualquier p, hice que los números dependan **del bloque y no del hilo**: `saltar` lleva cada bloque a su lugar en la lista de `rand_r`, y así cada bloque usa siempre los mismos números, sin importar qué hilo lo ejecute.

2. Problemas, diagnóstico y evidencia.

Problema	Cómo lo noté (evidencia)	Qué hice
Semilla equivocada	Usé la del enunciado (20261018); el profesor aclaró que es el código de cada uno	La cambié a 2224510 y repetí todo
Generador compartido (carrera)	Con `rand()` y 8 hilos el conteo cambió en cada corrida y tardó 11 s contra 1,6 s	Cada bloque con su propio estado
Misma semilla en los hilos	Con 8 hilos $\pi = 3{,}142351$, igual al de solo $N/8$ puntos	Cada bloque empieza en otro lugar (`saltar`)
Contador sin `reduction`	Con 1 hilo daba bien; con 8, π = 0,654; 0,694 y 0,640	`reduction(+:dentro)`
Semillas "distintas" que no lo eran	En la Parte I, las semillas $2224510 + 1000k$ daban un error medio 2,7 veces menor que el teórico: sus listas se solapaban	Tramos sin solapamiento con `saltar`; el error coincidió con la fórmula
Mediciones con ruido	Una corrida de 12,4 s (lo normal era 1,25 s); una lectura de frecuencia de 544 MHz con el procesador ocupado	Mediana y muestras de frecuencia más largas

3. Validación. Con $N = 10^8$ obtuve $\pi = 3{,}141668$, con un error (0,0000756) menor que el típico esperado (0,000164). Para la reproducibilidad exigí que el **conteo fuera idéntico**, no solo parecido: 78 541 707 en el secuencial, con 1, 2, 4 y 8 hilos, con `OMP_NUM_THREADS` y en las 35 corridas de la Tabla de la Parte III. Además verifiqué que el estado inicial de cada hilo con `saltar` coincide con el de `rand_r` avanzando uno por uno, y `montecarlo.c` compila sin advertencias con `-Wall -Wextra`.

4. Conexión con mis datos. Con $p = 4$ los siete tiempos fueron de 0,3449 a 0,3879 s y la **mediana quedó en 0,3700 s**, por eso $S(4) = 1{,}2549/0{,}3700 = 3{,}39$ y no 4. Crear los hilos cuesta solo 0,57 ms (0,15 %) y la frecuencia bajó apenas un 3 %; probé `schedule(dynamic)` por si unos hilos esperaban a otros y no mejoró (0,456 contra 0,412 s), así que el resto lo atribuyo a que Windows y WSL usan los mismos núcleos. Con $p = 8$, $S(8) = 5{,}02$, por debajo de Amdahl con $f = 0{,}940$ (5,64): ahí sí hay causas medidas: crear 8 hilos costó 13,6 ms (5,4 %), la frecuencia bajó un 15 % y hay 2 hilos por núcleo. La mediana también fue clave: la corrida de 12,41 s habría hecho que el promedio diera un $S(4)$ imposible de 7,7.

5. Reflexión. Aprendí que la reproducibilidad hay que diseñarla desde el principio: atar los números aleatorios al bloque y no al hilo la dio para cualquier p. También que un error de concurrencia puede esconderse (sin `reduction` acierta con un hilo), que dos semillas distintas no garantizan números distintos, y que medir en un portátil tiene ruido: hay que repetir y usar la mediana. En un próximo trabajo confirmaría primero los datos asignados (la semilla), usaría un generador hecho para trabajar en paralelo y sin el límite de 7×10^8 puntos, y fijaría cada hilo a un núcleo (`OMP_PROC_BIND`) para separar mejor el efecto del hiperprocesamiento.

Anexo. Bitácora de IA

Herramienta	Propósito	Qué aportó	Qué acepté o descarté y por qué
Claude Code (27/10/2026)	Responder la Parte I	Programas de apoyo y figuras	Acepté apoyar cada respuesta con un experimento real; descarté una versión con semillas correlacionadas
Claude Code (27/10/2026)	Programar la Parte II	Programa paralelo y programas de prueba de cada error	Acepté el diseño por bloques; cambié la semilla del enunciado por mi código
Claude Code (27/10/2026)	Medir la Parte III	Scripts de medición y gráficas	Acepté medir cada factor; descarté una lectura de frecuencia errónea
Claude Code (27/10/2026)	Redactar la Parte IV	Borrador del informe	Acepté la estructura; revisé que cada cifra coincida con mis salidas